1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

Register R0				Register R1			
00	00	00	12	FF	FF	FF	80
31		16 15	0	31		16 15	0

Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

Adresse der Konstanten:				Inhalt Register R2 nach Anw-03:			
08	00	01	A4	12	34	56	78
31		16 15	0	31		16 15	0

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

Adresse v. VariableA (Reg. R0 nach Anw-05)				Inhalt Register R1 nach Anw-05:			
20	00	00	00	00	00	12	34
31		16 15	0	31		16 15	0

Anzahl von Anw-05 geladener Bits: 16

Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:			
47	11	12	34
31		16 15	0

Anzahl von Anw-06 geladener Bits: 32

Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

Adresse v. VariableC:				Inhalt von VariableC nach Anw-07:			
20	00	00	04	47	11	12	34
31		16 15	0	31		16 15	0

10

Anzahl von Anw-07 gespeicherter Bits: 32

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Anw-16
Anw-17
Anw-18
Anw-19

G1 G2 G3 G4

dezimale Interpretation der Bitmuster: signed / unsigned

Reg. R1: 0 0 0 1 0 0 1 0 | 0 0 1 1 0 1 0 0 | 1 0 1 0 1 1 1 0 | 1 1 1 1 0 0 0 0 — 305419896 / 305419896

Reg. R2: 1 0 0 1 1 1 0 1 | 1 1 0 0 1 0 1 0 | 1 0 1 1 0 0 1 1 | 0 0 0 0 1 1 1 0 — −1647831194 / 2647294102

Reg. R3: 1 0 1 0 1 1 1 1 | 1 1 1 1 1 1 1 1 | 1 0 1 0 1 1 1 1 | 1 1 1 1 1 1 1 0 — −1342263298 / 2952703998

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N X Z

Ergebnis: Ok X / Nicht Ok

Ergebnis: Ok X / Nicht Ok

Anw-20
Anw-21
Anw-22

G1 G2 G3 G4

dezimale Interpretation der Bitmuster: signed / unsigned

Reg. R4: 1 1 0 0 1 1 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — −872415232 / 3422552064

Reg. R5: 0 1 0 0 1 1 1 1 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — 1308622848 / 1308622848

Reg. R6: 0 1 1 1 1 1 1 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — 2113929216 / 2113929216

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V X C X N Z

Ergebnis: Ok / Nicht Ok X

Ergebnis: Ok X / Nicht Ok

Anw-23
Anw-24
Anw-25

G1 G2 G3 G4

dezimale Interpretation der Bitmuster: signed / unsigned

Reg. R7: 0 0 1 0 0 1 1 1 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — 654311424 / 654311424

Reg. R8: 0 1 0 0 0 1 0 1 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — 1157627904 / 1157627904

Reg. R9: 1 1 1 0 0 0 1 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 | 0 0 0 0 0 0 0 0 — −503316480 / 3791650816

31 16 15 0

Kreuzen Sie die gesetzten Bits des Statusregisters an: V C N X Z

Ergebnis: Ok X / Nicht Ok

Ergebnis: Ok / Nicht Ok X

3 *Zugriff auf Felder*

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

MeinHalbwortFeld: Anfangsadresse: 20 00 00 14 (31 … 16 15 … 0) — Anzahl Elemente: 6 — Anzahl Bytes pro Element: 2

MeinWortFeld: Anfangsadresse: 20 00 00 20 (31 … 16 15 … 0) — Anzahl Elemente: 6 — Anzahl Bytes pro Element: 4

MeinTextFeld: Anfangsadresse: 20 00 00 38 (31 … 16 15 … 0) — Anzahl Elemente: 9 — Anzahl Bytes pro Element: 1

Anw-08 *Anw-09* *Anw-10* Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für das jeweilige Element erfolgen.

Register R1 nach Anw-09: 00 00 00 22 (31 … 16 15 … 0)

Register R2 nach Anw-10: 00 00 00 3E (31 … 16 15 … 0)

Anw-11 *Anw-12* Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:

Register R4 nach Anw-12: 00 00 00 4S (31 … 16 15 … 0)

Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements: 5

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13: 20 00 00 16 (31 … 16 15 … 0)

Register R5 nach Anw-13: 00 00 00 3E (31 … 16 15 … 0)

Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements: 1

Anw-14

Register R0 nach Anw-14: 20 00 00 18 (31 … 16 15 … 0)

Register R6 nach Anw-14: 00 00 FF CC (31 … 16 15 … 0)

Nr. (beginnend mit 0) des von Anw-14 gelesenen Elements: 2

Anw-15

Register R0 nach Anw-15: 20 00 00 1A (31 … 16 15 … 0)

Adresse des von Anw-15 überschriebenen Elements: 20 00 00 1A (31 … 16 15 … 0)

Nr. (beginnend mit 0) des von Anw-15 überschriebenen Elements: 3